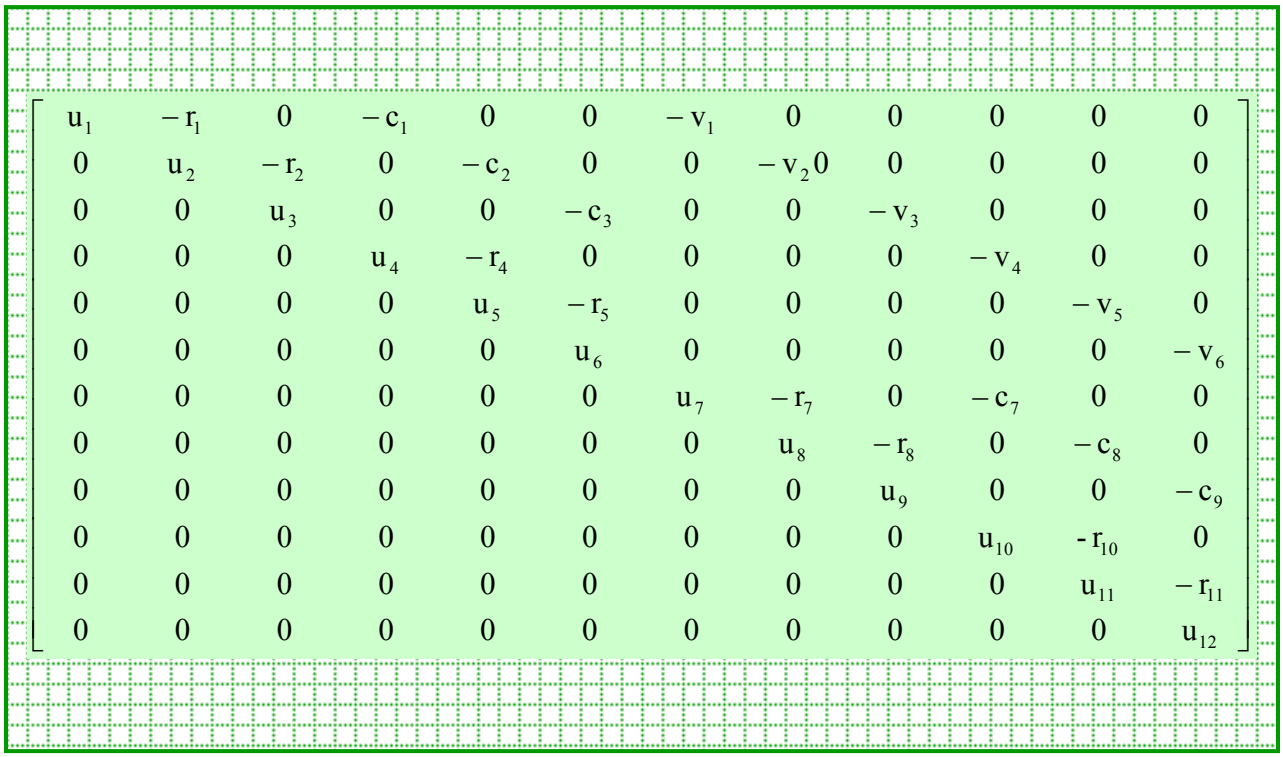
USGS
science for a changing world

PRECONDITIONED CONJUGATE-GRADIENT 2 (PCG2), A COMPUTER PROGRAM FOR SOLVING GROUND-WATER FLOW EQUATIONS

Water-Resources Investigations Report 90-4048



U.S. Department of the Interior
U.S. Geological Survey

PRECONDITIONED CONJUGATE-GRADIENT 2 (PCG2), A COMPUTER PROGRAM FOR SOLVING GROUND-WATER FLOW EQUATIONS

By Mary C. Hill

U.S. GEOLOGICAL SURVEY

Water-Resources Investigations Report 90-4048

Denver, Colorado
1990

Second Printing, 2003.
Significant changes were made on pages 8, 10 to 13, and 14.
The FORTRAN code is omitted, so the report now has 25 pages instead of 43 pages.

U.S. DEPARTMENT OF THE INTERIOR
MANUEL LUJAN, JR., *SECRETARY*

U.S. GEOLOGICAL SURVEY
DALLAS L. PECK, *DIRECTOR*

For additional information
write to:

Chief, Branch of Regional Research
U.S. Geological Survey
Box 25046, Mail Stop 418
Federal Center
Denver, CO 80225-0046

Copies of this report can
be purchased from:

U.S. Geological Survey
Books and Open-file Reports Section
Box 25425
Federal Center
Denver, CO 80225-0425

CONTENTS

ABSTRACT 1
INTRODUCTION 1
Purpose and Scope 1
Previous Investigations 2
Notation 3
THREE-DIMENSIONAL GROUND-WATER FLOW MODEL 3
SOLUTION BY THE PRECONDITIONED CONJUGATE-GRADIENT METHOD 4
MICCG 8
POLCG 11
Convergence Criteria 12
INPUT INSTRUCTIONS 13
Explanation of Fields Used in Input Instructions 13
Sample Data Inputs 14
Linking PCG2 to the Modular Model 15
DOCUMENTATION OF PCG2 16
Brief Description of Modules 16
Flowchart 16
Narrative for Modules 19
PCG2AL 19
PCG2RP 19
PCG2AP 19
SPCG2P 19
SPCG2E 19
Adapting SPCG2E for Computers with Vector and Parallel Architecture 20
List of Variables 21
REFERENCES 24

FIGURES

Figure 1.--Aquifer system volumes accounted for by conductances r_n, c_n, and v_n in the finite-difference method. 5
Figure 2.--Matrix U for MICCG. The r_n, c_n, and v_n are the conductances along rows and columns and between layers, respectively. 9
Figure 3.--Matrix $M = U^T D U$ for MICCG. The r_n, c_n, and v_n are the conductances along rows and columns and between layers, respectively. For this example $\delta = 0.0$ and $\alpha = 1.0$ in equation 11a. 10

PRECONDITIONED CONJUGATE-GRADIENT 2 (PCG2), A COMPUTER PROGRAM FOR SOLVING GROUND-WATER FLOW EQUATIONS

.By Mary C. Hill .

ABSTRACT

This report documents PCG2: a numerical code to be used with the U.S. Geological Survey modular three-dimensional, finite-difference, ground-water flow model commonly known as MODFLOW. PCG2 uses the preconditioned conjugate-gradient method to solve the equations produced by the model for hydraulic head. Linear or nonlinear flow conditions may be simulated

PCG2 includes two preconditioning options: modified incomplete Cholesky preconditioning, which is efficient on scalar computers; and polynomial preconditioning, which requires less computer storage and, with modifications that depend on the computer used, is most efficient on vector computers. Convergence of the solver is determined using both head-change and residual criteria. Nonlinear problems are solved using Picard iterations.

This documentation provides a description of the preconditioned conjugate-gradient method and the two preconditioners, detailed instructions for linking PCG1 to the modular model, sample data inputs, a brief description of PCG2, and a FORTRAN listing.

INTRODUCTION

Purpose and Scope

Finite-difference numerical models commonly are used to investigate ground-water flow systems. Effective use of these models requires that the matrix equations they produce be solved efficiently, that is, that a correct solution is produced using as little computer processing time as possible. Effective use of the models also requires that the amount of computer storage be minimized to allow for solution on small computers and to avoid over-burdening large computers

The purpose of this work is to document PCG2, a numerical code which uses the preconditioned conjugate-gradient method to solve the matrix equations produced by the U.S. Geological Survey modular three-dimensional, finite-difference ground-water flow model. Two preconditioning options are included which have not previously been available for use with the model, and which perform better than available solvers for many problems

Previous Investigations

Matrix equations have been solved using direct or iterative methods. In most direct methods the matrix is factored exactly and the true solution is obtained by executing one backward and one forward substitution. In most iterative methods an initial estimate of the solution is refined iteratively using an approximately factored matrix, and successive solutions should approach the true solution. Solution convergence is assumed to have been reached when some measure of the residual and(or) the difference in results between successive iterations is less than some user-specified convergence criteria. Direct solution is straightforward, but iterative methods are less susceptible to round-off error, are more efficient for large problems, and require less computer storage (Remson, Hornberger and Molz, 1971, p. 177; Aziz and Settari, 1979, p. 261).

The preconditioned conjugate-gradient method (Coneus, Golub and O'Leary, 1976) is an iterative method which can be used to solve matrix equations if the matrix is symmetric (matrix element $a_{ij} = a_{ji}$, where the first subscript is the matrix-row number, and the second is the matrix column number) and positive-definite (all eigenvalues are positive) (see Hildebrand, 1965, p. 30 and 48 for further discussion of these terms). The matrix produced in ground- water-flow models always is symmetric and positive-definite. The preconditioned conjugate-gradient method has been the subject of considerable interest in recent years because of its efficiency and ability to solve difficult problems (Meijerink and van der Vorst, 1977). It works well, in part, because the required iteration parameters are calculated internally and need not be estimated.

Various preconditioners may be used in the preconditioned conjugate-gradient method. Among the different preconditioners there often is a direct relationship between increased efficiency and increased computer storage (Meijerink and van der Vorst, 1977). To avoid this tradeoff, the only preconditioners considered are those that produce a solver that has computer storage requirements less than or equal to the strongly implicit procedure (SIP) as programmed for the ground-water flow problem. SIP requires additional computer storage equal to four arrays with dimensions equal to the number of grid nodes (McDonald and Harbaugh, 1988, chap. 12).

The incomplete Cholesky preconditioner (ICCG) has been very popular (Meijerink and van der Vorst, 1977; Kuiper, 1981, 1987). However, alternative methods of matrix preconditioning have been developed to achieve more efficient conjugate-gradient solvers. Axelsson and Lindskog (1986) presented a preconditioner that commonly is called the modified incomplete Cholesky preconditioner (MICCG). It is similar to preconditioners presented by Dupont and others (1968), Gustafsson (1978), Wong (1979), and Ashcraft and Grimes (1988). In this paper, MICCG refers to Axelsson and Lindskog's (1986) method. Hill (1990) showed that MICCG was more efficient than ICCG in solving eight ground-water flow test cases on a scalar computer. Saad (1985) presented a least-squares polynomial preconditioner (POLCG), in which the matrix inverse is approximated by a truncated Neuman polynomial series. It is similar to preconditioners presented by Dubois and others (1979) and Johnson and others (1983). In this paper POLCG refers to Saad's (1985) method. POLCG is not as efficient as SIP or HICCG on

scalar computers (Scandrett, 1989; Hill, 1990), but is more efficient on vector computers (Scandrett, 1989; Meyer and others, 1989, p. 1445). Storage requirements are less than for SIP: POLCG requires additional storage equal to three arrays with dimensions equal to the number of grid nodes.

Many preconditioners were excluded from PCG2. Modifications of ICCG presented by Meijerink and van der Vorst (1977; 1981), Gustafsson (1978; 1979), Wong (1979), and Ashcraft and Grimes (1988) apparently converge in fewer iterations than ICCG or MICCG. These were not included in PCG2 because they require that several additional arrays with dimensions equal to the number of grid nodes be added to computer storage. Additional polynomial preconditioners have been presented in several papers (Saad, 1985; Ashby, 1987; Meyer and others, 1989). Based on Saad's results, POLCG was chosen because it appears to be at least as efficient as the other polynomial methods and it is easier to use. However, for very large grids (greater than 100,000 cells) the optimal Chebyshev polynomial preconditioner (Meyer and others, 1989) may be more efficient, and users with large grids may wish to consider this alternative polynomial preconditioner.

Watts (1981) and Hill (1990) indicate that the greatest differences in solver efficiency on scalar computers occur for three-dimensional, non-linear problems. Thus, for these types of problems, it may be well worth the time and effort to try more than one solver. SIP generally is a good alternative to consider.

Notation

The following notation is used in this work:

Underlined capital letters indicate matrices: $\underline{A}$

Underlined lower-case letters indicate column vectors: $\underline{r}$

The element located in matrix row i and column j is designated as follows: matrix $\underline{A}$, element a_{ij}

Exception: a single index is used to simplify notation in some cases. These are described in the text.

THREE-DIMENSIONAL GROUND-WATER FLOW MODEL

In this work, selected numerical methods are presented for solving the matrix equations that arise when the finite-difference method is used to discretize the ground-water flow equation as applied to a two-dimensional aquifer or a three-dimensional layered aquifer system. The finite-difference model is described in detail by McDonald and Harbaugh (1988), and only aspects relevant to the preconditioned conjugate gradient solver are discussed here.

The finite-difference model produces a set of linear equations which can be expressed in matrix notation as:

$$\underline{A}\,\underline{x} = \underline{b} \qquad (1)$$

where A is a coefficient matrix which is discussed below, x is a vector of hydraulic heads at each grid cell, and b is a vector of defined flows, terms associated with head-dependent boundary conditions, and storage terms (for transient problems) at each grid cell. Because of the rigid structure of the finite-difference grid, and because there are six neighboring cells to each internal cell of a three-dimensional grid, A is symmetric and there may be as many as six off-diagonals in A--three above the main diagonal and three below. The elements on the off-diagonals equal the negatives of the horizontal or vertical conductances between the centers of the cells which make up the finite-difference grid (fig. 1). The horizontal conductances along columns (c_n of fig. 1) equal $T_i T_{i+1} \Delta w / (T_i \Delta L_{i+1} / 2 + T_{i+1} \Delta L_i / 2)$, where T_i, and T_{i+1}. are transmissivities between two adjoining cells, Δw is the width of the two cells, and ΔL_i ΔL_{i+1} are lengths of the two cells. Horizontal conductances along rows (r_n of fig. 1) are defined analogously. The vertical conductances (v_n of fig. 1) equal $K_z A / \Delta_z$, where K_z is the vertical hydraulic conductivity, A is the area of the cell, and A_z is the vertical distance between the centers of the two cells. Each component on the main diagonal of A, a_{ii}, equals:

$$a_{ii} = \sum_{j=1, i \neq j}^{N} (-a_{ij}) + w_i \qquad (2)$$

where N is the total number of nodes in the grid; a_{ij} are the off-diagonal elements of row *i*, which are negative numbers; and w_i are the sum of the conductances associated with head-dependent boundaries and storage terms (for transient problems), which are positive numbers. Besides being symmetric, A is also positive-definite (its eigenvalues are always positive) (Varga, 1962, p. 23, 181-188; Hildebrand, 1965, p. 48), and these properties allow equation 1 to be solved using the methods presented in this work.

Nonlinearities occur if any aquifer is unconfined or if a head-dependent boundary condition is nonlinear. If any aquifer is unconfined, the horizontal conductances are a function of hydraulic head, and the main-diagonal and four of the six off-diagonals of matrix A must be updated during the solution process. If a head-dependent boundary condition is nonlinear, the boundary condition may change from being head-dependent to defined flux depending on the hydraulic head in the aquifer adjacent to the boundary, and the main diagonal of A and vector b (eq. 1) must be updated.

SOLUTION BY THE PRECONDITIONED CONJUGATE-GRADIENT METHOD

The preconditioned conjugate-gradient method for solving a set of linear equations is iterative. In iterative methods, it is assumed that the matrix A can be split into the sum of two matrices; that is A = M + N (Varga, 1962, p. 87-93; Remson and others, 1971, p. 177). M is called the preconditioned form of A, and the goal is to define M so that it is easy to invert and resembles A as much as possible. These two criteria generally are impossible to satisfy simultaneously, and the optimal definition of M has been the focus of much research. In the preconditioned conjugate-gradient method, M must always be symmetric and positive definite. (This brief description of matrix splitting does not include important requirements that M and N must satisfy to achieve a convergent solver. Please refer to Varga (1962) for additional information).

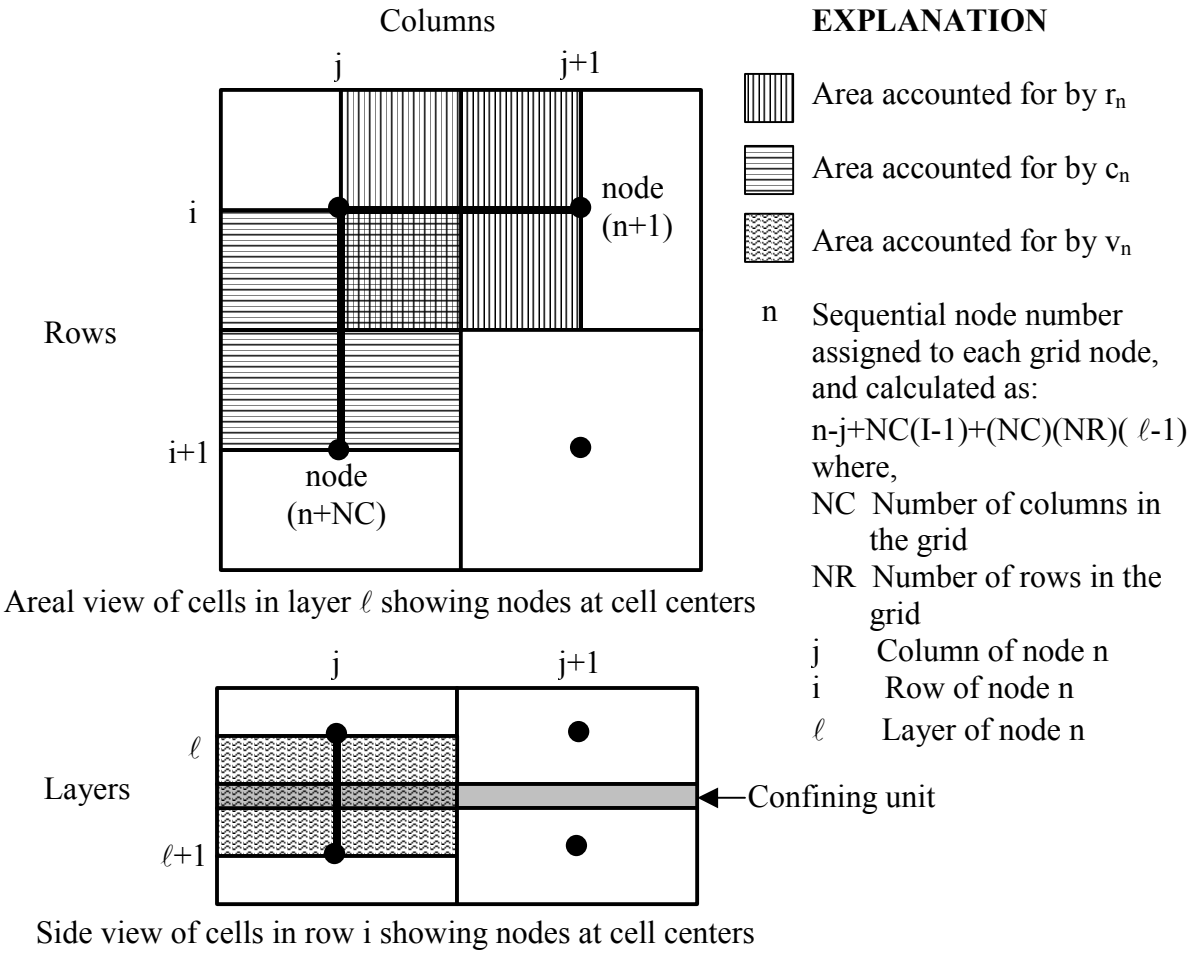


Figure 1.--Aquifer system volumes accounted for by conductances r_n, c_n, and v_n in the finite-difference method.

Once $\underline{M}$ has been defined, the basic iterative equation is developed from equation 1 and the splitting of $\underline{A}$, and can be written as:

$$\underline{M}\,\underline{x}_{k+1} = \underline{M}\,\underline{x}_k + \underline{b} - \underline{A}\,\underline{x}_k \qquad (3)$$

where k is the iteration index. Noting that $\underline{b} - \underline{A}\,\underline{x}_k$ is the residual ($\underline{r}_k$) of the original set of equations at the kth iteration, and setting $\underline{s}_k = \underline{x}_{k+1} - \underline{x}_k$ gives

$$\underline{M}\,\underline{s}_k = \underline{r}_k \qquad (4)$$

or

$$\underline{s}_k = \underline{M}^{-1} \underline{r}_k \qquad (5)$$

The new heads may then be calculated as $\underline{s}_{K=1} = \underline{x}_k + \underline{s}_k$. More generally, some function of $\underline{s}_k$ may be used to calculate $\underline{x}_{k+1}$.

Conjugate-gradient methods are second-order iterative techniques because at each iteration the new change in $\underline{x}$, which is called $\underline{p}_k$ is calculated using the change from the prior iteration, $\underline{p}_{k\text{-}1}$, in addition to the vector $\underline{s}_k$ of equation 5. Conjugate-gradient methods begin by calculating $\underline{r}_o = \underline{b} - \underline{A}\,\underline{x}_o$. The following steps are executed for each iteration, starting with k = 0:

$$\underline{s}_k = \underline{M}^{-1} \underline{r}_k \qquad (6a)$$

for k= 0 $\qquad \underline{p}_k = \underline{s}_k \qquad (6b)$

for k > 0

$$\begin{cases} \beta_k = \dfrac{\underline{s}_k^T \underline{r}_k}{\underline{s}_{k-1} \underline{r}_{k-1}} & (6b) \\ \underline{p}_k = \underline{s}_k + \beta_k \underline{p}_{k-1} & (6c) \end{cases}$$

$$\alpha_k = \frac{\underline{s}_k^T \underline{r}_k}{\underline{p}_k^T \underline{A}\, \underline{p}_k} \qquad (6e)$$

$$\underline{x}_{k+1} = x_k + \alpha_k \underline{p}_k \qquad (6f)$$

$$\underline{r}_{k+1} = \underline{r}_k + \alpha_k \underline{A}\, \underline{p}_k \qquad (6g)$$

where the superscripted T indicates the transpose of the vector. Because $\underline{r}_{k+1}$ can be calculated using the last statement, $\underline{b}$ need not be saved within the solver. Iteration parameters β_k and α_k are calculated internally such that successive updating vectors, $\underline{p}_k$ are $\underline{A}$-orthogonal to previous $\underline{p}_k$ vectors -- that is, $\underline{p}_k^T \underline{A}\, \underline{p}_\ell = 0, k \neq \ell$ (Hestenes and Stiefel, 1952).

Whether or not an iterative method will converge and, if so, how fast depends on the preconditioner and how $\underline{x}_k$ is updated. A discussion of convergence is beyond the scope of this report, but references are cited for the convergence properties of each solver. See Varga (1962) for the general theory of convergence of iterative methods.

Scaling of the matrix $\underline{A}$ simplifies POLCG (Dubois and others, 1979), and is accomplished as:

$$\underline{B} = \underline{S}^T \underline{A}\, \underline{S} \qquad (7)$$

where $\underline{B}$ is the scaled matrix, all off-diagonal entries of $\underline{S}$ are zero, and

$$s_{ii} = \frac{1}{\sqrt{a_{ii}}} . \quad (8)$$

This type of scaling is called diagonal scaling, and it preserves the symmetry of the original matrix. Diagonal scaling may improve the matrix characteristics that are most important to convergence because the scaled matrix is still symmetric and positive definite, and the condition number of $\underline{A}$ (the largest eigenvalue divided by the smallest eigenvalue) is minimized (Forsythe and Strauss, 1955). However, although $\underline{A}$ is diagonally dominant because:

$$a_{ii} \geq \sum_{\substack{j=1 \\ i \neq j}}^{N} \left| a_{ij} \right| , \quad (9)$$

$\underline{B}$ may not be diagonally dominant if some of the values along the diagonal of $\underline{A}$ are much smaller than others. For example, consider the following original and scaled matrices:

$$\underline{A} = \begin{bmatrix} 0.9 & -0.1 & -0.75 \\ -0.1 & 0.2 & 0.0 \\ -0.75 & 0.0 & 0.8 \end{bmatrix} \qquad \underline{B} = \begin{bmatrix} 1.0 & -0.23 & -0.88 \\ -0.23 & 1.0 & 0.0 \\ -0.88 & 0.0 & 1.0 \end{bmatrix}$$

The first row of $\underline{B}$ is no longer diagonally dominant. The lack of diagonal dominance can cause problems when using MICCG. Scaling also may cause more rounding and truncation errors because all components of the diagonal of $\underline{A}$ must be summed, as in equation 2. Without scaling, the conductance terms can be manipulated individually to reduce rounding and truncation errors (Dorn and McCracken, 1972, p. 94). Scaling was only used for POLCG.

The precision with which numbers are stored in the computer can significantly affect solver performance. For example, making the four arrays required by SIP double precision (14 to 15 significant digits on the computer used) instead of single precision (6 to 7 significant digits) can make the difference between convergence and nonconvergence for some problems (McDonald and Harbaugh, 1988, p. A-2; A.W. Harbaugh, U.S. Geological Survey, written commun., 1989). However, increasing the precision of arrays doubles the required computer storage space. All the arrays required by the solvers presented in this work were declared as single precision. Double-precision scalar variables were used to improve the accuracy of calculations, where possible. The only double-precision array in the model is, then, the array used to store calculated heads (McDonald and Harbaugh, 1988, p. A-2). In PCG2, problems caused by the limited precision of the solver arrays are most prevalent for POLCG (Hill, 1990). If the limited precision of the solver arrays is suspected as the cause of convergence problems, arrays V, SS, and P may be converted to double precision by doubling their allocated storage in PCG2AL, and declaring them as double precision in PCG2AP and SPCG2E

Nonlinear problems are solved by Picard iterations, in which the matrix $\underline{A}$ and vector $\underline{b}$ are periodically updated between iterations using the newly calculated heads. For nonlinear problems, convergence using the conjugate-gradient solvers was found to be most efficient if several iterations (called inner iterations in this report) were accomplished between Picard iteration updates (Kuiper, 1981 and 1987). This allows the solver to use equations 6c and 6d to

calculate several orthogonal $\underline{p}_k$ vectors before updating $\underline{A}$ and $\underline{b}$, and thus take advantage of the orthogonality of the conjugate-gradient method. The total number of iterations equals the sum of the inner iterations for all updates of $\underline{A}$ and $\underline{b}$. For any one $\underline{A}$ and $\underline{b}$, the inner iterations continue until one of the following occurs: (1) the user-defined maximum number of inner iterations (ITERI of the input file) are executed; or (2) the final convergence criteria are met. Outer iterations continue until the final convergence criteria are met on the first inner iteration after an update. The total number of iterations required is minimized by adjusting ITER1 and re-executing the problem. For most problems, the optimal value of ITERI ranges from 3 to 10, though much larger values have been useful for some problems (L.F. Konikow, U.S. Geological Survey, oral commun., 2001).

In the absence of round-off error, only inner iterations would be required for linear problems. However, in practice, round-off error may adversely affect the residual calculated by the conjugate-gradient method (eq. 6g) when more than 50 iterations are required. Recalculating the residual as $\underline{r} = \underline{b} - \underline{A}\,\underline{x}$ occasionally by limiting the number of inner iterations to less than 50 in linear problems alleviates the error. This can be accomplished using ITER1 of the input file.

The conjugate-gradient method fails if $\underline{r}_k = 0$ for any iteration k. This was found to occur when the Drain Package was used and the hydraulic head in the drain cell was set equal to or less than the level of the drain. Thus, the drain was not represented in the finite-difference equation first used in internal preconditioned conjugate-gradient iterations, and $\underline{r}_k$ was equal to zero at the second internal iteration. To accommodate this circumstance, subroutine PCG2AP checks for $\underline{r}_k = 0$. If it is detected, the PCG2AP response depends on the value of MXITER, the maximum number of external solver iterations. MXITER commonly is set to 1 if the problem is linear.

1. If MXITER = 1, the following error message is printed and execution is stopped: CONJUGATE-GRADIENT METHOD FAILED. SET MIXITER GREATER THAN ONE AND TRY AGAIN. STOP EXECUTION.
2. If MXITER > 1, a new outer iteration is initiated using the hydraulic heads from the end of the first inner iteration.

A flowchart which displays most of the steps discussed above is presented in the section "Documentation of PCG2" of this report.

MICCG

In modified incomplete Cholesky preconditioning, $\underline{M} = \underline{U}^T D\, U$, where U is an upper triangular matrix with nonzero values along the main diagonal and at off-diagonal locations where $\underline{A}$ has nonzero values. $\underline{D}$ is a positive diagonal matrix with $d_{ii} = 1/u_{ii}$. When A is structured as in the finite-difference model with natural ordering of the nodes and there are more than two columns in the grid, the off-diagonal components of $\underline{U}$ equal the off-diagonal components of $\underline{A}$. That is, $u_{ij} = a_{ij}$, for $j > i$. As an example, figure 2 shows $\underline{U}$ for a problem with 2 rows, 3 columns and 2 layers. To more clearly indicate the physical quantities involved, the variables r_n, c_n , and v_n which are depicted in figure 1 and were described earlier in this paper, are used. To be consistent, the same subscript, n, is used for the diagonal of matrix $\underline{U}$, so that u_{ii} now becomes u_n, where $n = i$.

Calculation of the u_{ii} is explained by executing the matrix multiplication for the simple problem shown in figure 2.

$$\begin{bmatrix}
u_1 & -r_1 & 0 & -c_1 & 0 & 0 & -v_1 & 0 & 0 & 0 & 0 & 0 \\
0 & u_2 & -r_2 & 0 & -c_2 & 0 & 0 & -v_2 & 0 & 0 & 0 & 0 \\
0 & 0 & u_3 & 0 & 0 & -c_3 & 0 & 0 & -v_3 & 0 & 0 & 0 \\
0 & 0 & 0 & u_4 & -r_4 & 0 & 0 & 0 & 0 & -v_4 & 0 & 0 \\
0 & 0 & 0 & 0 & u_5 & -r_5 & 0 & 0 & 0 & 0 & -v_5 & 0 \\
0 & 0 & 0 & 0 & 0 & u_6 & 0 & 0 & 0 & 0 & 0 & -v_6 \\
0 & 0 & 0 & 0 & 0 & 0 & u_7 & -r_7 & 0 & -c_7 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & u_8 & -r_8 & 0 & -c_8 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & u_9 & 0 & 0 & -c_9 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & u_{10} & -r_{10} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & u_{11} & -r_{11} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & u_{12}
\end{bmatrix}$$

Figure 2.--Matrix $\underline{U}$ for MICCG. The r_n, c_n, and v_n are the conductances along rows and columns and between layers, respectively.

In matrix $\underline{U}^T \underline{D} \underline{U}$ (fig. 3), $-r_n$, $-c_n$, and $-v_n$ appear in the same places they occupied in the $\underline{A}$ matrix. The additional off-diagonal terms occur because $\underline{U}^T \underline{D} \underline{U}$ is an incomplete factorization of $\underline{A}$. The u_n are defined such that the sum of the elements along a row of $\underline{U}^T \underline{D} \underline{U}$ equals the sum of the elements along the same row of $\underline{A}$. To accomplish this for the matrix shown in figure 3:

$$u_1 = a_{11},$$

$$u_2 = a_{22} \frac{r_1^2}{u_1} - \frac{r_1 c_1}{u_1} - \frac{r_1 v_1}{u_1},$$

$$u_3 = a_{33} \frac{r_2^2}{u_2} - \frac{r_2 c_2}{u_2} - \frac{r_2 v_2}{u_2}, \qquad (10)$$

etc.

The general algorithm can be expressed as (R.L. Cooley, 1992, p. 80):

$$u_{ii} = (1+\delta)a_{ii} - \sum_{k=1}^{i-1} \frac{u_{ki}^2}{u_{kk}} - \alpha \sum_{j=1}^{i-1} f_{ji} + \sum_{j=i+1}^{N} f_{ij} \qquad (11a)$$

u_1	$-r_1$	0	$-c_1$	0	0	$-v_1$	0	0	0	0	0
	$u_2+\xi_1$	$-r_2$	φ_1	$-c_2$	0	ϕ_1	$-v_2$	0	0	0	0
		$u_3+\xi_2$	0	φ_2	$-c_3$	0	θ_2	$-v_3$	0	0	0
			u_4+r_1	$-r_4$	0		0	0	$-v_4$	0	0
				$u_5+\xi_4+\tau_2$	$-r_5$	0	ϕ_2	0	θ_4	$-v_5$	0
					$u_6+\xi_5+\tau_3$	0	0	ϕ_3	0	θ_5	$-v_6$
						$u_7+\gamma_1$	$-r_7$	0	$-c_7$	0	0
		symmetric					$u_8+\xi_7+\gamma_2$	$-r_8$	φ_7	$-c_8$	0
								$u_9+\xi_8+\gamma_3$	0	φ_8	$-c_9$
									$u_{10}+\tau_7\gamma_4$	$-r_{10}$	0
										$u_{11}+\xi_{10}+\tau_8+\gamma_5$	$-r_{11}$

$$\xi_n = \frac{r_n^2}{u_n} \qquad \tau_n = \frac{c_n^2}{u_n} \qquad \gamma_n = \frac{v_n^2}{u_n} \qquad \varphi_n = \frac{r_n c_n}{u_n} \qquad \theta_n = \frac{r_n v_n}{u_n} \qquad \phi_n = \frac{c_n v_n}{u_n}$$

Figure 3.--Matrix $\underline{M} = \underline{U}^T \underline{D} \underline{U}$ for MICCG. The r_n, c_n, and v_n are the conductances along rows and columns and between layers, respectively. For this example = 0.0 and = 1.0 in equation 11a.

where

$$f_{ij} = \begin{cases} \sum_{k=1}^{i=1} u_{ki} u_{kj} u_{kk} & a_{ij} = 0 \\ \\ = 0 & a_{ij} \neq 0 \end{cases}, \qquad (11b)$$

and

$$u_{ij} = 0 \ \ for \ \ j < i \ .$$

Again, the more general notation for components of $\underline{U}$ is used. The f_{ji} and f_{ij} are equivalent to the ψ_n, θ_n, and ϕ_n of figure 3, and they occur off the main diagonal of $\underline{U}^T \underline{D}\ \underline{U}$.

The variable α is a user-defined relaxation parameter and is used to diminish the value of the f_{ij} of equation lla. Ashcraft and Grimes (1988) found that using a relaxation parameter value of 0.97, 0.98, or 0.99 instead of 1.00 sometimes improved convergence by as much as 50 percent. However, consistently using a value of 1.00 for the relaxation parameter generally produces solutions that are at least as efficient as those attained with other commonly used solvers (Hill, 1990). The variable δ is calculated internally, as described later in this section.

The equation $\underline{U}^T \underline{D}\ \underline{U}\ s_k = r_k$ is solved by a two-step process. First $\underline{U}^T \underline{D}\ v_k = r_k$ is solved for v_k by forward substitution, then $\underline{D}\ \underline{U}\ \underline{s}_k = \underline{D}\ \underline{v}_k$ is solved for $\underline{s}_k$ by backward substitution.

The MICCG preconditioned matrix presented in this paper is from Axelsson and Lindskog (1986), and is nearly identical to that presented by Dupont and others (1968), Gustafsson (1978, eq. 3.1, and 1979, eq. 6), Wong (1979), and Ashcraft and Grimes (1988). In the method used in this report, the overcompensation parameter, δ, used by Gustafsson (1978) to augment a_{ii} of equation lla usually equals zero. Ashcraft and Grimes (1988) found that the number of iterations required to achieve convergence was insensitive to values of the overcompensation factor, so using a value of zero should not affect convergence. An exception occurs, however, when equation lla produces values of u_{ii} that are zero or negative. Manteuffel (1980, p. 483) suggests that these circumstances can be accommodated by making δ of equation lla a positive number. Cooley (1992, p. 80) suggests that if equation lla produces values of u_{ii} that are zero, or negative, δ can be increased using the empirically derived algorithm $\delta_{new} = 1.5 \times \delta_{old} + 0.001$ until all u_{ii} values are positive. This algorithm is used in PCG2.

Convergence properties of incomplete Cholesky with row-sums agreement are discussed by Gustafsson (1978).

POLCG

In Neuman series polynomial preconditioning, $\underline{M}^{-1}$ equals the sum of several terms of a power-series expansion for the inverse of matrix $\underline{A}$ (Dubois and others, 1979; Johnson and others, 1983; Saad, 1985), so that

$$\underline{M}^{-1} = \underline{I} + \underline{A} + \underline{A}^2 + \cdots + \underline{A}^{\prime} \qquad (12)$$

Then, by weighting the terms as suggested by Johnson and others (1983) and Saad (1985), an approximate solution can be written as

$$\underline{s}_k = \underline{M}^{-1}\,\underline{r}_k = c_0 : \underline{r}_k + c_1\,\underline{A}\,\underline{r}_k + c_2.\,\underline{A}^2\,\underline{r}_k + \underline{A}^3\,\underline{r}_k \qquad (13)$$

when $l = 3$, and c_0, c_l and c_2 are coefficients chosen to optimize convergence. For computational efficiency, equation 13 is calculated by the following series of steps:

$$\begin{aligned} \underline{z}_1 &= c_2\ \underline{r}_k + \underline{A}\,\underline{r}_k \\ \underline{z}_2 &= c_1\,\underline{r}_k + \underline{A}\,z_1 \\ \underline{s}_k &= c_0\,\underline{r}_k + \underline{A}\,\underline{z}_2 \end{aligned} \qquad (14)$$

so that the powers of $\underline{A}$ are never formed explicitly (Dubois and others, 1979, p. 259). One of the advantages of POLCG is that the steps of equation 14 are efficient on vector and parallel computers.

The coefficients can be calculated using the method described by Saad (1985, p. 869-871; 880) as: $c_0 = \frac{-15}{32}g^3$, $c_1 = \frac{27}{16}g^2$, $c_2 = \frac{-9}{4}g$ -15 where g is the upper bound on the maximum eigenvalue of A, estimated as the largest sum of the absolute values of the components in any row of $\underline{A}$ (Varga, 1962, p. 17; Gerschgorin, 1931). For a scaled matrix, g is generally close to 2. Scandrett (1989) and Hill (1990) used g=2, and the number of iterations required to achieve solutions in their test cases were generally insensitive to changes in g. Using NBPOL of the input file, the user can specify that g=2 or that g is to be estimated as described above. Estimation of g uses slightly more execution time per iteration.

Convergence properties of polynomial preconditioners are discussed by Saad (1985).

Convergence Criteria

An iterative matrix solver is assumed to have converged when some measure of the residual and(or) the difference in results between successive iterations is less than user-specified convergence criteria. In PCG2, the difference between results of successive iterations is measured using both the maximum absolute value of the change in hydraulic head and the maximum absolute value of the all-by-all residual for that iteration. Typical values for these error criteria are 0.01 ft and 0.01 ft^3/s, respectively. The convergence criteria are dimensional, so different values are required when different units are used. For example, if the time unit is days instead of seconds, the equivalent maximum absolute value of cell-by-cell residual for that iteration would be 864 ft^3/d.

The defined convergence criteria are too large if the global ground-water flow budget errors calculated by the modular model (McDonald and Harbaugh, 1988, p. 3-16 to 3-22) are unacceptably large. What is unacceptable depends on the problem being considered and must be determined by the user. For most ground-water flow problems, global budget errors greater than one percent are unacceptable. If unacceptably large global budget errors occur, the error criteria should be reduced.

The defined convergence criteria are too small if the accuracy achieved by the solver exceeds the accuracy required by the user. For example, Hill (1990) found that reducing both error criteria from 10^{-3} to 10^{-6} increased the execution time by as much as 55 percent, and, especially for POLCG, resulted in lack of convergence in some test cases. If the solver is taking more iterations than expected to achieve convergence and the calculated global budget error is smaller than required by the user, or if the solver is not converging and the convergence criteria are very small, the user can increase the convergence criteria.

INPUT INSTRUCTIONS

In MODFLOW-2000, input for PCG2 is read from a file that is type "PCG" in the name file. All numeric variables are free format if the option "FREE" is specified in the Basic Package input file; otherwise, all variables have 10-character fields. The input for PCG2 is as follows.

FOR EACH SIMULATION

0. [#Text]
Item 0 is optional. The "#" symbol needs to be in column 1. Item 0 can be repeated many times.

l. Data: MXITER ITER NPCOND

2 Data: HCLOSE RCLOSE RELAX NBPOL IPRPCG MUTPCG DAMP

Explanation of Fields Used in Input Instructions

MXITER----is the maximum number of outer iterations -- that is, calls to the solution routine. For a linear problem MXITER should be 1, unless more than 50 inner iterations are required, when MXITER could be as large as 10. A larger number (generally less than 100) is required for a nonlinear problem.

ITERI--------is the maximum number of inner iterations. For nonlinear problems, ITERI usually ranges from 3 to 10; a value of 30 will be sufficient for most linear problems.

NPCOND---is the flag used to select the matrix preconditioning method. The following options are available.

NPCOND	PRECONDITIONING METHOD
1	Modified Incomplete Cholesky (for use on scalar computers)
2	Polynomial (for use on parallel computers)

HCLOSE----is the head change criterion for convergence, in units of length. When the maximum absolute value of the head change at all nodes during an iteration is less than or equal to HCLOSE, and the criterion for RCLOSE is satisfied (see below), iteration stops. Commonly, HCLOSE equals 0.01.

RCLOSE----is the residual criterion for convergence, in units of cubic length per time. When the maximum absolute value of the residual at all nodes during an iteration is less than or equal to RCLOSE, and the criterion for HCLOSE is satisfied (see above), iteration stops. Commonly, RCLOSE equals HCLOSE.

For nonlinear problems, convergence is achieved when the convergence criteria are satisfied on the first inner iteration.

RELAX------is the relaxation parameter used with NPCOND=L (MICCG). Usually, RELAX=1.0, but for some problems a value of 0.99, 0.98, or 0.97 will reduce the number of iterations required for convergence. RELAX is not used if NPCOND≠I.

NBPOL------is used when NPCOND=2 to indicate whether the estimate of the upper bound on the maximum eigenvalue is 2.0, or whether the estimate will be calculated. NBPOL=2 is used to specify the value as 2.0; for any other value of NBPOL, the estimate is calculated. Convergence is generally insensitive to this parameter. NBPOL is not used if NPCOND does not equal 2.

IPRPCG-----is the printout interval for PCG. If IPRPCG is equal to zero, it is changed to 999. The extreme head change and residual (positive or negative) are printed for each iteration of a time step whenever the time step is an even multiple of.IPRPCG. The printout also occurs at the end of each stress period regardless of the value of IPRPCG.

MUTPCG - is a flag that controls printing of convergence information from the solver:

0 - print tables of maximum head change and residual each iteration

1 - print only the total number of iterations

2 – print nothing from the solver

3 - print as for 0, but only if solver convergence fails

DAMP-------is a damping parameter. Values less than l.0 can be used to aid convergence in nonlinear problems.

Sample Data Inputs

```
#Example data for a linear problem with fewer than about 50 iterations
#Time in seconds, length in feet.
1    50    1                                  Item 1
.01    .01    1.    2    0    2    0.0       Item 2
```

```
#Example data set for a nonlinear problem or a linear problem with more than about 50 iterations
#Time in seconds, length in feet.
50    5    1                                  Item 1
.01    854.    2    0    1    0.0            Item 2
```

LINKING PCG2 TO THE MODULAR MODEL

LAYCON=3 layers may produce a matrix which is not diagonally-dominant when using the BCF package as presented in McDonald and Harbaugh (1988, p. 5-22 and 5-59). MICCG (NPCOND=I) of the PCG2 package will not function if the matrix is not diagonally dominant, and POLCG may not converge. To correct this, make the following change in module BCFlFM (M.G. McDonald, U.S. Geological Survey, written commun., 1989):

McDonald and Harbaugh (1988) version:

```
C7D-----WITH HEAD BELOW TOP ADD CORRECTION TERMS TO RHS AND HCOF.
      RHS(J,I,K)=RHS(J,I,K) + CV(J,I,K-I)*TOP(J,I,KT)
      HCOF(J,I,K)=HCOF(J,I,K) + CV(J,I,K-1)
  220 CONTINUE
```

Modified version:

```
C7D-----WITH HEAD BELOW TOP ADD CORRECTION TERMS TO RHS AND HCOF.
C7D-----MODIFIED TO PUT CORRECTION COMPLETELY ONTO RIGHT HAND SIDE
      RHS(J,I,K)=RHS(J,I,K) + CV(J,I,K-I)*(TOP(J,I,KT)-HTMP) 220
      CONTINUE
```

DOCUMENTATION OF PCG2

Brief Description of Modules

Three primary modules and two submodules were created for the preconditioned conjugate-gradient method. The following is a brief description of the purpose of each of these modules:

Primary modules:

PCG2AL	Allocates space for the conjugate-gradient calculations.
PCG2RP	Reads, stores and prints the input data.
PCG2AP	Performs multiple iterations of the conjugate-gradient method and checks the convergence criteria.

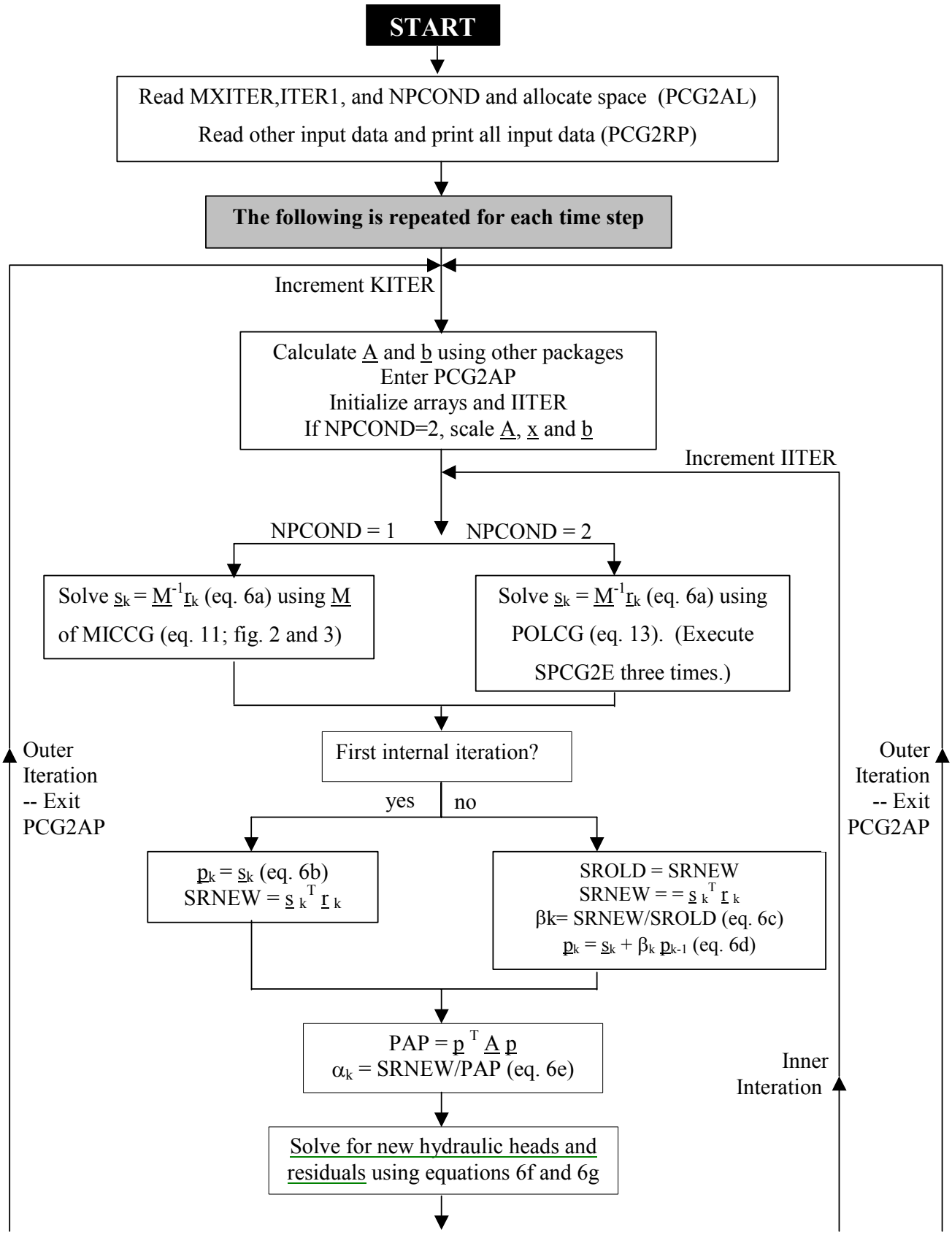
Submodules:

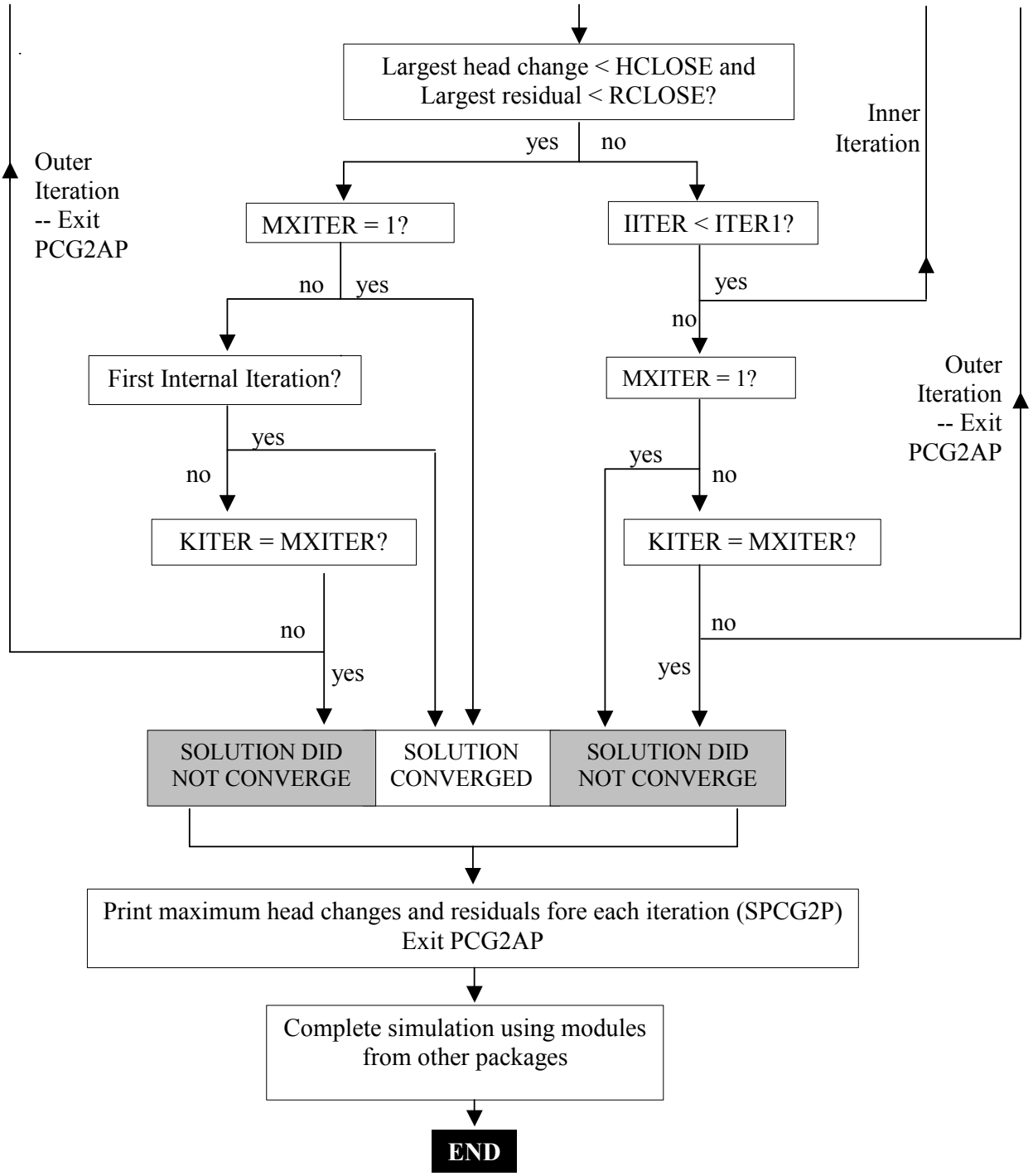
SPCG2P	Called by PCG2AP to print the extreme head changes and residuals that occurred at each iteration.
SPCG2E	Called by PCG2AP to perform one matrix multiplication required by the polynomial preconditioner. This submodule is executed three times for each iteration of POLCG.

Flowchart

The flowchart for the package is shown below, and includes the functions performed by the three Primary modules and the two submodules. The following variable names used in the flowchart are taken from the FORTRAN code. Some were defined in the input instructions; all are defined later in the following list of variables.

HCLOSE, IITER, ITERI, MXITER,
NPCOND, PAP, RCLOSE, SRNEW, SROLD


START
Read MXITER,ITER1, and NPCOND and allocate space (PCG2AL)
Read other input data and print all input data (PCG2RP)
The following is repeated for each time step
Increment KITER
Calculate A and b using other packages
Enter PCG2AP
Initialize arrays and IITER
If NPCOND=2, scale A, x and b
Increment IITER
NPCOND = 1
NPCOND = 2
Solve sk = M-1rk (eq. 6a) using M of MICCG (eq. 11; fig. 2 and 3)
Solve sk = M-1rk (eq. 6a) using POLCG (eq. 13). (Execute SPCG2E three times.)
First internal iteration?
yes
no
Outer Iteration -- Exit PCG2AP
Outer Iteration -- Exit PCG2AP
pk = sk (eq. 6b)
SRNEW = s k T r k
SROLD = SRNEW
SRNEW = = s k T r k
βk= SRNEW/SROLD (eq. 6c)
pk = sk + βk pk-1 (eq. 6d)
PAP = p T A p
αk = SRNEW/PAP (eq. 6e)
Inner Interation
Solve for new hydraulic heads and residuals using equations 6f and 6g


Largest head change < HCLOSE and
Largest residual < RCLOSE?
yes
no
Inner
Iteration
Outer
Iteration
-- Exit
PCG2AP
MXITER = 1?
IITER < ITER1?
no
yes
yes
no
First Internal Iteration?
MXITER = 1?
Outer
Iteration
-- Exit
PCG2AP
yes
yes
no
no
KITER = MXITER?
KITER = MXITER?
no
no
yes
yes
SOLUTION DID
NOT CONVERGE
SOLUTION
CONVERGED
SOLUTION DID
NOT CONVERGE
Print maximum head changes and residuals fore each iteration (SPCG2P)
Exit PCG2AP
Complete simulation using modules
from other packages
END

Narrative for Modules

PCG2AL

Module PCG2AL reads MXITER, ITERl, and NPCOND, and allocates space in the X array for the arrays required for the solver. Arrays SS, P and V are required for both values of NPCOND; array CD is required only for NPCOND=L. Each of these four arrays are vectors dimensioned equal to the number of grid nodes. Additionally, four smaller arrays are required to store the maximum head change and residual at each iteration (HCHG and RCHG) and the cell locations where these occurred (LHCH and LRCH).

PCG2RP

Module PCG2RP reads HCLOSE, RCLOSF,, RELAX, NBPOL, IPRPCG, MUTPCG, and IPCGCD, and prints all variables read for this package.

PCG2AP

Module PCG2AP performs up to ITERl iterations of the preconditioned conjugate-gradient algorithm for solving the flow equation. To save computational time, all arrays are declared one dimensional. They are accessed by a single index which is calculated from the layer, row, and column indexes normally used to access the arrays in three dimensions.

Double precision scalar variables are used for most calculations in this module to improve the accuracy of the results. Modification of the present use of double precision may affect simulated results.

For the polynomial preconditioner, c_0 and c_l of equation 14 are negative in the text, but are calculated as positive numbers in the FORTRAN code. The FORTRAN code is correct because, as programmed in the modular model, matrix A of equation l is negative definite instead of positive definite. While this poses no mathematical difficulty, it does require that the odd-numbered coefficients of equation 14 be positive instead of negative.

The steps executed by PCG2AP were outlined in the flowchart previously presented in this section.

SPCG2P

Submodule SPCG2P prints the extreme values of the head change (HCHG) and residual (RCHG) out of all cells for each iteration of a time step. The cell location (LHCH and LRCH) where the values occur also is printed.

SPCG2E

Submodule SPCG2E calculates the matrix-vector multiplication and the vector addition required by each of the three parts of equation 14. It is called three times for each polynomial iteration.

Adapting SPCG2E for Computers with Vector and Parallel Architecture

Use of the polynomial preconditioner on computers with vector and(or) parallel architecture will be most efficient if SPCG2E is modified to take advantage of the computer used. The following points should be considered when modifying SPCG2E.

The diagonal entries of A all equal -1.0 because A is scaled and is negative definite (see narrative for module PCG2AP). Vectors CR, CC, and CV contain the off-diagonals of A along which nonzero entries occur. Note that although array CV is dimensioned using NODES in SPCG2E and elsewhere in the model, it is originally only given storage space in X for cells in NLAY-1 layers (McDonald and Harbaugh, 1988, p. 4-25). In making changes, care must be taken to avoid overwriting elements of array HCOF, which is stored in X after CV.

Using w to represent r_k, z_l, or z_2, and n to represent the cell number, the nth element of each vector produced by the matrix-vector multiplications of equation 14 is calculated by summing -w(n) with:

CR(n) * w(n+l)	(15a)
CR(n) * w(n)	(15b)
CC(n) * w(n+NCOL)	(15c)
CC(n) * w(n)	(15d)
CV(n) * w(n+NRC) n•NODES-NRC	(15e)
CV(n) * w(n)	(15f)

Equations 15a through 15f are vector-vector multiplications that can be calculated quickly on computers with vector architecture, but one problem exists. Along with rows and columns for active cells, A includes rows and columns for inactive and constant-head cells. Inactive cells are accounted for by setting appropriate entries of CR, CC, and CV to zero in the beginning of the "DO ll5" loop in PCG2AP and, therefore, cause no problem. However, constant-head cells are accounted for using the IF statements in SPCG2E, and these IF statements must be eliminated to vectorize the multiplications.

One way to eliminate the IF statements in SPCG2E is to add a work vector of length NODES (allocate space in PCG2AL) and use this vector to store CR, CC, or CV temporarily while performing the multiplications of equation 15 with the work vector. In the work vector, the entries along rows associated with constant-head cells can be set to zero prior to performing the multiplications. Thus, in equation 15a, CR(n)=0 if cell n+l is constant head; in equation 15c, CC(n)=0 if cell n+NCOL is constant head; in equation 15e, CV(n)=0 if cell n+NRC is constant head. In equations 15b, 15d, and 15f, CR(n)=0, CC(n)=0, and CV(n)=0 if cell n is constant head.

Alternatively, vectors CR, CC, and CV may be used directly in equation 15 and the values that are set to zero may be stored in a separate array and replaced once the multiplication has been completed. This would require a work vector with length equal to the number of constant-head cells in the grid. Presently, the number of constant-head cells is not available when space is allocated by calling PCG2AL, and the user would have to modify the data input and the module to provide this information.

List of Variables

Variables for the entire package are listed below. Variables not listed below are used only briefly in a few calculations and their meaning can be identified from nearby lines of the code. The name of the module is listed under 'Range' for variables used by only one module.

Variable	Range	Definition
ALPHA	PCG2AP	Double-precision field containing α_k of equation 6e.
BIGH	PCG2AP	Value of head change with the largest absolute value for one iteration.
BIGR	PCG2AP	Value of residual with the largest absolute value for one iteration.
BPOLY	PCG2AP	Estimated upper bound of the maximum eigen value of A; used in polynomial preconditioning.
C0, Cl, C2	PCG2AP	Scalar coefficients of equations 13 and 14.
CC	Global	DIMENSION (NCOLNROW,NLAY), conductance along columns (see fig. 1).
CD	PCG2AP	DIMENSION (NCOL,NROW,NLAY), u_{ii} of equation lla.
CDl	PCG2AP	The first nonzero component of CD. Used to ensure that all values in CD are all ≥ 0 or ≤ 0.
CDCC, CDCR, CDCV	PCG2AP	Double-precision fields containing u_{ki}^2 / u_{kk} terms of equation lla.
CR	Global	DIMENSION (NCOL,NROW,NLAY), conductance along rows (see fig. 1).
CV	Global	DIMENSION (NCOL,NROW,NLAY), conductance between layers (see fig. 1).
DONE	Package	Double-precision field containing a one.
DZERO	PCG2AP	Double-precision field containing a zero.
FCC, FCR, FCV	PCG2AP	Double-precision field containing f_{ij} terms of equation ll.
HCHG	Package	DIMENSION (MXITER*ITERI), extreme head change (BIGH) for each iteration.
HCHGN	PCG2AP	Double-precision field containing the head change at one cell at one iteration.
HCLOSE	Package	Closure criteria for the head change for the iterative procedure.
HCOF	Global	DIMENSION (NCOL,NROW,NLAY), coefficient of head at cell (J,I,K) in the finite-difference equation.
HNEW	Global	DIMENSION (NCOL,NROW,NLAY), most recent estimate of head in each cell. HNEW changes at each iteration.
IBOUND	Global	DIMENSION (NCOL,NROW,NLAY), status of each cell <0, constant-head cell =0, inactive cell >0, variable-head cell
ICNVG	Global	Flag set equal to zero until the iteration procedure has converged, when it is set to one.
IICNVG	PCG2AP	Inner iteration convergence flag.
IITER	PCG2AP	Inner iteration counter. Reset each time PCG2AP is called.
IN	Package	Primary unit number from which input for this package will be read.

Variable	Range	Definition
IOUT	Global	Primary unit number for all printed output. IOUT=6.
IPRPCG	Package	Frequency (in time steps) with which the extreme head changes and residuals for each iteration will be printed.
ISIZ	PCG2AL	Number of cells (nodes) in the finite-difference grid.
ISOLD	Package	Before this module allocates space, ISOLD is set equal to ISUM. After allocation, ISOLD is subtracted from ISUM to get ISP, the amount of space in the X array allocated by this module.
ISP	PCG2AL	Number of words in the X array allocated by this module.
ISUM	Global	Index number of the lowest element in the X array which has not yet been allocated. When space is allocated for an array, the size of the array is added to ISUM.
ITERl	Package	Maximum number of inner iterations.
KITER	Global	Counts the number of times PCG2AP is called.
KPER	Global	Stress period counter.
KSTP	Global	Time step counter. Reset at the start of each stress period.
LCname	Package	Location in the X array of the first element of array ‘name’.
LENX	Global	Length of the X array in words. This should always be equal to the dimension of X specified in the MAIN program.
LHCH	Package	DIMENSION (3,MXITER*ITERl), Layer, row, and column of the cell containing the extreme residual (BIGH) for each iteration.
LRCH	Package	DIMENSION (3,MXITER*ITERl), Layer, row, and column of the cell containing the extreme residual (BIGR) for each iteration
MXITER	Package	Maximum number of calls of PCG2AP.
MUTPCG	Package	Flag to control printing from the solver '(see input instructions).
N	Package	Cell index.
NBPOL	Package	Used when NPCOND=2 to indicate how the value of the upper bound of the maximum eigenvalue is calculated (see input instructions).
NCD	Package	One-dimensional subscript of conductance to the adjacent cell, which is in the last column.
NCF	Package	One-dimensional subscript of conductance to the adjacent cell, which is in the next column.
NCL	Package	One-dimensional subscript of the cell index of the adjacent cell which is in the last column.
NCN	Package	One-dimensional subscript of the cell index of the adjacent cell which is in the next column.
NCOL	Global	Number of columns in the grid.
NITER	PCG2AL	Counts the total number of inner iterations that are executed.
NLAY	Global	Number of layers in the grid.
NLL	Package	One-dimensional subscript of the cell index of the adjacent cell which is in the last layer.
NLN	Package	One-dimensional subscript of the cell index of the adjacent cell which is in the next layer.

Variable	Range	Definition
NLS	Package	One-dimensional subscript of conductance to the adjacent cell which is in the next layer.
NLZ	Package	One-dimensional subscript of conductance to the adjacent cell which is in the last layer.
NODES	Global	Number of cells (nodes) in the finite-difference grid.
NORM	PCG2AP	Flag for scaling the matrix set of equations. NORM=L and scaling is executed only when NPCOND=2.
NPCOND	Package	Preconditioner used: 1 Incomplete Cholesky with row-sums agreement 2 Polynomial
NRB	Module	One-dimensional subscript of conductance to the adjacent cell which is in the last row.
NRC	Package	Number of cells in a model layer.
NRH	Package	One-dimensional subscript of conductance to the adjacent cell which is in the next row.
NRL	Package	One-dimensional subscript of the cell index of the adjacent cell which is in the last row.
NRN	Package	One-dimensional subscript of the cell index of the adjacent cell which is in the next row.
NROW	Global	Number of rows in the grid.
NSTP	Global	Number of time steps in the current stress period.
P	PCG2AP	DIMENSION (NCOL,NROW,NLAY), p_k and p_{k-l} of equations 6d-6g.
PAP	PCG2AP	Double-precision field containing the denominator of equation 6e.
RCHG	PCG2AP	DIMENSION (MXITER*ITERl), Extreme residual (BIGR) for each iteration.
RCHGN	PCG2AP	Double-precision field containing the residual change at one cell at one iteration.
RCLOSE	Package	Closure criteria for the residual for the iterative procedure.
RELAX	Package	Relaxation parameter of equation lla.
RHS	Global	DIMENSION (NCOL,NROW,NLAY), right-hand side of the finite-difference equation. RHS is an accumulation of terms from several different packages.
SROLD	PCG2AP	Double-precision field containing the denominator of equation 6c.
SRNEW	PCG2AP	Double-precision field containing the numerator of equations 6c and 6e.
SS	PCG2AP	DIMENSION (NCOL,NROW,NLAY), $\underline{s}_k$ of equation 6a-6e.
V	PCG2AP	DIMENSION (NCOL,NROW,NLAY), intermediate solution when solving equation 6a, and when calculating PAP.

REFERENCES

Ashby, S.F., 1987, Polynomial preconditioning for conjugate gradient methods: Department of Computer Science, Report UIUCDCS-R-87-1355, University of Illinois, Urbana-Champaign, IL., 131 p.

Ashcraft, C.C., and Grimes, R.G., 1988, On vectorizing incomplete factorization and SSOR preconditioners: SIAM Journal of Scientific and Statistical Computing, v. 9, no. 1, p. 122-151.

Axelsson, O., and Lindskog, G., 1986, On the eigenvalue distribution of a class of preconditioning methods: Numerical Mathematics, v. 48, p. 479-498.

Aziz, Khalid and Settari, Antonin, 1979, Petroleum reservoir simulation, Elsevier, 476 p.

Cooley, R.L., 1992, A modular finite-element method (MODFE) for areal and axisymetric ground-water flow problems, part 2, Derivation of finite-element equations and comparisons with analytical solutions: U.S. Geological Survey Techniques of Water-Resources Investigations Bk. 6, ch. A4, 108 p.

Concus, Paul**,** Golub, G.H., and O'Leary, D.P., 1916, A generalized conjugate gradient method for the numerical solution of elliptic partial differential equations, in Bunch, J.P., and Rose, D.J., eds., Sparse Matrix computations, Academic Press, P. 303-332.

Dorn, W.S. and McCracken, D.D., 1972, Numerical methods with FORTRAN IV case studies: John Wiley, 447 p.

Dubois, P.F., Greenbaum, A., Rodrique, G.H., 1979, Approximating the inverse of a matrix for use in iterative algorithms on vector processors: Computing, vol. 22, p. 257-268.

Dupont, Todd, Kendall, R.P., and Rachford, H.H., Jr., 1968, An approximate factorization procedure for solving self-adjoint elliptic difference equations: SIAM Journal on Numerical Analysis, v. 5, no. 3, p. 559-573.

Forsythe, G.E. and Strauss, E.G., 1955, On best conditioned matrices:

American Mathematical Society Proceedings, vol. 6, no. 3, p. 340-345. Gerschgorin, S., 1931, Ner die abrenzung der eigenwerte einer matrix: Isv. Akad. Nauk SSSR Ser. Mat., vol. 7, p. 749-754.

Gustafsson, Ivar, 1978, A class of first order factorization methods: BIT v. 18, p. 142-156.

_____1979, On modified incomplete Cholesky factorization methods for the solution of problems with mixed boundary conditions and problems with discontinuous material coefficients: International Journal for Numerical Methods in Engineering, v. 14, p. 1127-1140.

Hestenes, M.R. and Stiefel, Eduard, 1952, Methods of conjugate gradients for solving linear systems: Journal of Research for the National Bureau of Standards, vol. 49, no. 6, p. 409-436.

Hildebrand, F.B., 1965, Methods of applied mathematics: Prentice-Hall, Inc., 362 p.

Hill, M.C., l990, Solving ground-water flow problems by conjugate-gradient methods and the strongly implicit procedure: Water Resources Research, v. 26, no. 9, p. 1061-1969.

Johnson, O.G., Micchelli, C.A., and Paul, George, 1983, Polynomial preconditioners for conjugate gradient calculations: SIAM Journal of Numerical Analysis, v. 20, no. 2, p. 362-376.

Kuiper, L.K., 1981, A comparison of the incomplete Cholesky-conjugate gradient method with the strongly implicit method as applied to the solution of two-dimensional groundwater flow equations: Water Resources Research, v. 17, no.4, p. 1082-1086.

_____1987, Computer program for solving ground-water flow equations by the preconditioned conjugate gradient method: U.S. Geological Survey Water- Resources Investigations Report 87-4091, 34 p.

Manteuffel, T.A., 1980, An incomplete factorization technique for positive definite linear systems: Mathematics of computation, vol. 34, no. 150, p. 473-497.

McDonald, M.G., and Harbaugh, A.W., 1988, A modular three-dimensional ground- water flow model: U.S. Geological Survey Techniques of Water-Resources Investigations, bk. 6, ch. Al, 548 p.

Meijerink, J.A., and van der Vorst, H.A., 1977, An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-Matrix: Mathematics of Computation, v. 31, no. 137, p. 148-162.

_____1981, Guidelines for the usage of incomplete decompositions in solving sets of linear equations as they occur in practical problems, Journal of Computational Physics, v. 44, p. 134-155.

Meyer, P.D., Valocchi, A.J., Ashby, S.F., and Saylor, P.E., 1989, A numerical investigation of the conjugate gradient method as applied to three- dimensional groundwater flow problems in randomly heterogeneous porous media: Water Resources Research, vol., 25, no. 6, p. 1440-1446.

Remson, Irwin, Hornberger, G.M. and Molz, F.J., 1971, Numerical methods in subsurface hydrology: John Wiley, 389 p.

Saad, Y., 1985, Practical use of polynomial preconditionings for the conjugate gradient method: SIAM Journal of Scientific and Statistical Computing, v. 6, no. 4, p. 865-881.

Varga, R.S., 1962, Matrix iterative analysis: Prentice-Hall, Inc., 322 p. Watts, J. W., 1981 9 A conjugate gradient-truncated direct method for the iterative solution of the reservoir simulation pressure equation: Society of Petroleum Engineers, v. 21, no. 3, p. 345-353.

Wong, Y.S., 1979, Pre-conditioned conjugate gradient methods for large sparse matrix problems, in Proceedings of the First International Conference on Numerical Methods in Thermal Problems, Swansea, U.K., July 2-6, 1979, eds. R.W. Lewis and K. Morgan, p. 967-979.